

Chapter 7 - Vision Transformers

Updated Exam Study Guide - Theory Only, Fixed Equation Formatting, and Complete Flashcards

Main exam goal: understand how images become patch tokens, why positional and class tokens matter, how ViT encoder blocks work, and why ViTs connect naturally to modern generative and vision-language models.

What was updated: all formulas were rewritten in clean symbolic form; missing exam-relevant explanations were added smoothly into the matching sections; programming/code blocks were removed; all 50 flashcards were added as a final active-recall section.

Sources used: Chapter 7 Vision Transformers study guide, Chapter 7 full annotated lecture, and Chapter 7 flashcard deck.

1. What You Must Be Able to Explain

- Why a Vision Transformer turns an image into a sequence of visual tokens.
- How patch count, patch dimension, patch embeddings, and sequence length are computed.
- Why positional information is essential for image patches.
- How the class token works and why it is trainable, optional, and useful for classification.
- How self-attention, MLP/FFN, residual connections, and LayerNorm form a ViT encoder block.
- How classification differs from general-purpose image encoding.
- Why CNNs and ViTs have different strengths and tradeoffs.
- Why ViTs are important for CLIP/SigLIP, MaskGIT, U-ViT, DiT, and modern multimodal systems.

2. Clean Formula Sheet

Concept	Symbolic form
Patch count	$N = (H/P)(W/P) = HW/P^2$
Flattened patch dimension	$\dim(\text{patch}) = P^2C$
Patch-token matrix	$X_{\text{patch}} \in \mathbb{R}^{N \times D}$
Add positional embeddings	$X' = X + P_{\text{pos}}$
Class-token input sequence	$X_0 = [x_{\text{cls}}; x_1; x_2; \dots; x_N] \in \mathbb{R}^{(N+1) \times D}$
QKV projections	$Q = XW_Q, K = XW_K, V = XW_V$
Scaled dot-product attention	$\text{Attention}(Q, K, V) = \text{softmax}((QK^T)/\sqrt{d_h}) V$
Relative positional bias	$\text{score}_{ij} = (q_i k_j^T)/\sqrt{d_h} + b_{(\text{delta_row}, \text{delta_col})}$
Pre-LN ViT block	$X' = X + \text{MSA}(\text{LN}(X)); X_{\text{next}} = X' + \text{MLP}(\text{LN}(X'))$

Concept	Symbolic form
MLP / FFN	$\text{MLP}(x) = W_2 \text{ sigma}(W_1 x + b_1) + b_2$
Classification head	$\text{logits} = W_{\text{cls}} h_{\text{cls}}^{(L)} + b$
Mean pooling alternative	$h_{\text{img}} = (1/N) \sum_{i=1}^N h_i^{(L)}$
Cross-entropy loss	$L = -\log \text{softmax}(\text{logits})_y = -\log p(y_{\text{true}} x)$
Pipeline shape	$[B, C, H, W] \rightarrow [B, N, D] \rightarrow [B, N+1, D] \rightarrow [B, D] \rightarrow [B, K]$
CLS-token update	$x_{\text{cls}} \leftarrow x_{\text{cls}} - \eta \text{ partial} L / \text{partial} x_{\text{cls}}$

3. Why Vision Transformers Matter

A Vision Transformer (ViT) reframes vision as sequence modeling. Instead of applying convolutional filters over pixels as the main computation, it splits an image into fixed-size patches, embeds each patch as a token, adds positional information, and processes the sequence with Transformer encoder blocks.

This is important because token-based processing makes vision compatible with the same architectural language used in text models. Once images are represented as visual tokens, they can be combined with text tokens, timestep embeddings, condition tokens, and latent variables in multimodal and generative systems.

Self-attention also gives global context: each patch token can directly interact with distant image regions. This helps coordinate object layout, symmetry, repeated patterns, and long-range image consistency.

```
image -> patches -> patch embeddings -> positional information -> Transformer encoder
-> task head
```

4. From Image Tensor to Patch-Token Sequence

For an image with height H , width W , and C channels, a ViT divides the image into non-overlapping patches of size $P \times P$. Each patch is flattened into a vector of length P^2C , then projected into a D -dimensional embedding.

The important exam idea is that the image is converted from a 2D spatial object into a sequence of N visual tokens. A patch is therefore similar to a word token in a language Transformer, but it represents a local image region instead of a word or subword.

Patch embedding can be understood as a learned linear projection applied to every patch. Conceptually, it maps raw patch pixels into a semantic token space that the Transformer can process.

$$N = (H/P)(W/P) = HW/P^2$$

$$\text{dim}(\text{patch}) = P^2C$$

$$X_{\text{patch}} \text{ in } \mathbb{R}^{N \times D}$$

5. Positional Embeddings

Self-attention by itself is permutation-invariant: if we shuffle tokens, the attention operation does not inherently know that the image layout changed. Therefore, ViTs must add positional information so that each patch carries information about where it came from.

Learned absolute positional embeddings are a simple and strong baseline for fixed-resolution classification. However, because images naturally lie on a 2D grid, 2D-aware approaches are often better for tasks that depend strongly on spatial relationships.

Relative positional bias encodes how two patches are located relative to each other, such as left, right, above, or below. Conditional positional encoding generates the position signal from the current token

neighborhood. RoPE injects position by rotating queries and keys before attention scores are computed.

$$X' = X + P_{\text{pos}}$$

$$\text{score}_{ij} = (q_i k_j^T) / \sqrt{d_h} + b_{(\text{delta_row}, \text{delta_col})}$$

$$\text{2D absolute example: } p_{(u,v)} = p_u^x + p_v^y$$

6. Class Token and Image-Level Prediction

For classification, ViT often prepends a learnable class token x_{cls} to the patch sequence. This token is not an image patch. It is a trainable vector whose role is to collect global information from all patch tokens through bidirectional self-attention.

The class token works because ViT classification uses encoder-style bidirectional attention rather than GPT-style causal attention. Since there is no causal mask, the class token can attend to all patches and become a compact global summary of the image.

The class token is updated through backpropagation from the classification loss, just like any other trainable parameter. It is optional: classification can also use mean pooling over final patch tokens.

$$X_0 = [x_{\text{cls}}; x_1; x_2; \dots; x_N]$$

$$\text{logits} = W_{\text{cls}} h_{\text{cls}}^{(L)} + b$$

$$h_{\text{img}} = (1/N) \sum_{i=1}^N h_i^{(L)}$$

$$x_{\text{cls}} \leftarrow x_{\text{cls}} - \eta \text{partial}L/\text{partial}x_{\text{cls}}$$

7. Transformer Encoder Block inside ViT

A ViT encoder block uses LayerNorm, multi-head self-attention, residual connections, and an MLP/FFN. Attention mixes information across patch positions, while the MLP transforms each token independently after context has been gathered.

Residual connections preserve the original token representation while adding the learned update. This improves gradient flow and makes deep Transformer stacks easier to optimize. LayerNorm stabilizes feature scale and reduces training instability.

The MLP/FFN usually expands the token dimension, applies a nonlinear activation such as GELU, and projects back to the original dimension. It changes the features inside each token but preserves the sequence shape.

$$Q = XW_Q, K = XW_K, V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}((QK^T) / \sqrt{d_h}) V$$

$$X' = X + \text{MSA}(\text{LN}(X))$$

$$X_{\text{next}} = X' + \text{MLP}(\text{LN}(X'))$$

$$\text{MLP}(x) = W_2 \text{sigma}(W_1 x + b_1) + b_2$$

8. Classification versus General-Purpose Encoding

The same ViT encoder can support different tasks. In classification, the final image representation is passed to a class head and produces K logits. The loss is usually cross-entropy.

In general-purpose encoding, the goal is not a class label but a reusable image embedding. That embedding can be used for retrieval, clustering, transfer learning, contrastive learning, or image-text alignment in CLIP/SigLIP-style models.

This distinction matters because the encoder backbone can remain similar while the head and training objective change.

$$\text{Classification: } \text{logits in } \mathbb{R}^K$$

$$\text{Encoding: } z_{\text{img}} = \text{pool}(H^{(L)}) \text{ or } \text{projection}(\text{pool}(H^{(L)}))$$

$$L = -\log \text{softmax}(\text{logits})_y$$

9. CNNs versus ViTs

CNNs contain strong local inductive biases: locality, translation-related structure, and hierarchical feature extraction. This often makes CNNs more data-efficient, especially when data is limited.

ViTs rely more on learned token interactions and global self-attention. They often shine when trained with large-scale data, strong pretraining, or when integrated into multimodal and generative systems.

The best exam answer should avoid saying that one architecture always beats the other. CNNs and ViTs make different tradeoffs, and many modern systems combine ideas from both.

$$\text{self-attention cost} \approx O(N^2D)$$

$$N = HW/P^2$$

smaller $P \rightarrow$ larger $N \rightarrow$ higher attention cost

10. Historical Development and Modern Systems

The original Transformer showed that sequence modeling can be built around self-attention. DETR showed that Transformer-style reasoning can work for object detection. ViT showed that patch tokens can support image classification.

DeiT improved data-efficient ViT training with distillation. Swin added hierarchical shifted local windows for dense prediction. MAE showed that ViTs are strong self-supervised encoders. CLIP and SigLIP connect ViT-style image encoders to text.

For generation, MaskGIT predicts masked visual tokens, U-ViT treats noisy patches, time, and conditions as tokens, and DiT uses a Transformer over latent patches in diffusion models.

ViT: patch tokens for classification

DeiT: distillation token

Swin: shifted local windows

MAE: masked patch reconstruction

CLIP/SigLIP: image-text alignment

DiT/U-ViT: Transformer-style generative backbones

11. Practical Tensor-Shape Pipeline - No Code

For a small image classifier, the image batch begins as $[B, C, H, W]$. Patch embedding converts it to $[B, N, D]$. Adding a class token changes the sequence length to $[B, N+1, D]$. After Transformer blocks, the class-token output or pooled patch output becomes $[B, D]$. The final classifier gives $[B, K]$ logits.

Training conceptually minimizes cross-entropy, and evaluation compares the predicted class argmax with the true label. This section is about the mathematical shape flow only; no programming details are needed for the exam review.

$$[B, C, H, W] \rightarrow [B, N, D] \rightarrow [B, N+1, D] \rightarrow [B, D] \rightarrow [B, K]$$

$$p(y=k \mid x) = \text{softmax}(\text{logits})_k$$

$$L = -\log p(y_{\text{true}} \mid x)$$

12. Sample Exam Questions with Strong Answers

Question 1: Consider a ViT without a class token. Is image classification still possible?

Answer: Yes. The model can aggregate final patch tokens using mean pooling, attention pooling, or another learned summarization method, then apply a classifier. The tradeoff is that the model loses a dedicated sequence position whose job is to collect task-specific global information.

Question 2: Why do residual connections and LayerNorm matter in deep ViTs?

Answer: Residual connections preserve information and improve gradient flow, while LayerNorm stabilizes feature scales. They do not directly encode visual structure, but without them deep Transformer stacks are much harder to train reliably.

Question 3: Why are positional embeddings essential in ViT?

Answer: Because self-attention alone is permutation-invariant. Without positional embeddings, the model knows the patch content but not its spatial location. Positional information tells the model where each patch belongs in the image grid.

Question 4: Why is the rise of Transformer backbones in image generation conceptually important?

Answer: It shows that Transformers are becoming general representation mechanisms, not only language models. The same token-based architecture can support understanding, image-text alignment, and generation by changing the objective and head/backbone usage.

Question 5: Summarize the full conceptual flow of a Vision Transformer.

Answer: The image is split into patches; patches are flattened and projected into embeddings; positional information is added; an optional class token is prepended; Transformer encoder blocks contextualize the token sequence; finally, a classification head or embedding head maps the representation to the desired output.

13. Complete Flashcard Review - 50 Questions

The flashcards below are kept in active-recall format. Programming/code blocks were removed; code-related cards were rewritten as theory or tensor-shape concepts so they remain useful for the exam without including implementation code.

Flashcard 1

Q: What is the central architectural idea behind a Vision Transformer (ViT)?

A: It splits an image into fixed-size patches and treats them as a sequence of visual tokens.

Flashcard 2

Q: Why is the token-based view of images important for modern generative models?

A: It allows models to condition, scale, and mix image data across different modalities such as text.

Flashcard 3

Q: In generative modeling, what advantage does global context provide through self-attention?

A: Every token has a direct path to every other token, enabling coordination of structure across the whole image.

Flashcard 4

Q: What did Diffusion Transformer (DiT) demonstrate compared with traditional U-Nets?

A: Replacing the diffusion U-Net with a Transformer backbone can be highly effective and can scale predictably with model size.

Flashcard 5

Q: Why are Vision Transformers highly compatible with multimodal systems such as text-to-image generators?

A: Both visual structure and language can be expressed and jointly processed in a unified token form.

Flashcard 6

Q: Which architecture synthesizes images by predicting masked visual tokens with a bidirectional Transformer?

A: MaskGIT.

Flashcard 7

Q: How does U-ViT treat noisy image patches, time, and conditions?

A: It treats them all as tokens within a single Transformer-style framework.

Flashcard 8

Q: The original 2017 Transformer paper showed that sequence modeling could be built entirely around what?

A: Self-attention.

Flashcard 9

Q: What significant signal did DETR provide to computer vision?

A: It showed that Transformer-style reasoning is useful for vision tasks such as object detection by reframing them as set-prediction problems.

Flashcard 10

Q: What task did the original ViT paper focus on?

A: Image classification using patch-based Transformers.

Flashcard 11

Q: What was the primary contribution of DeiT regarding training efficiency?

A: It introduced data-efficient ViT training and a distillation token for task-specific supervision.

Flashcard 12

Q: What architectural feature did Swin Transformer introduce for dense-prediction tasks?

A: A hierarchical structure using shifted local attention windows.

Flashcard 13

Q: What did the Masked Autoencoder (MAE) demonstrate about ViTs?

A: It showed that ViTs are strong encoders for self-supervised representation learning by reconstructing masked image patches.

Flashcard 14

Q: If an image has height H, width W, and patch size P, what is the number of patches N?

A: $N = HW/P^2$.

Flashcard 15

Q: What is the purpose of the linear projection step after patches are flattened?

A: It maps each flattened patch vector into a learned embedding space of dimension D.

Flashcard 16

Q: Why is it necessary to add positional embeddings to patch tokens?

A: Transformers are permutation-invariant and do not automatically know the spatial location of tokens.

Flashcard 17

Q: What is the role of the class token (CLS) in a Vision Transformer?

A: It collects global information from all patches through self-attention for use in classification.

Flashcard 18

Q: In a standard Transformer encoder block, which component typically follows the first LayerNorm step?

A: Multi-head self-attention.

Flashcard 19

Q: What is the function of the MLP head in a ViT designed for classification?

A: It takes the final class-token representation and produces the output logits for predicted classes.

Flashcard 20

Q: How does a general-purpose encoder differ from a classifier in terms of the final image embedding?

A: The encoder produces a single feature vector, through the class token or pooling, rather than class logits.

Flashcard 21

Q: True or False: the class token is a fixed, non-trainable vector prepended to the patch sequence.

A: False. It is trainable and updated during training.

Flashcard 22

Q: During training, how is the class token x_{cls} updated?

A: It is updated through backpropagation from the loss function, like any other learnable parameter.

Flashcard 23

Q: What architectural condition allows the class token to gather information from the whole image?

A: Bidirectional attention, meaning there is no causal mask blocking access to patch tokens.

Flashcard 24

Q: What is mean pooling in ViTs, and how does it relate to the class token?

A: It is an alternative to the class token where the final image representation is the average of all patch-token representations.

Flashcard 25

Q: What is the standard representation for adding absolute positional embeddings P to patch tokens E ?

A: $Z_0 = [x_{cls}; E] + P$.

Flashcard 26

Q: What is a major drawback of learned absolute positional encodings when the input image resolution changes?

A: The learned positional table usually has to be interpolated to match the new number of patches.

Flashcard 27

Q: How do relative positional encodings differ from absolute ones in vision tasks?

A: They encode how two patches are positioned relative to each other, such as above or left of, rather than fixed coordinates.

Flashcard 28

Q: What specific positional mechanism does Swin Transformer use?

A: Relative position bias added to the attention logits.

Flashcard 29

Q: What is the primary mechanism of Conditional Positional Encoding (CPE)?

A: The positional signal is generated dynamically from the current token neighborhood using a Position Encoding Generator (PEG).

Flashcard 30

Q: How does Rotary Positional Embedding (RoPE) apply position information?

A: By rotating the queries and keys in the attention mechanism before computing the scores.

Flashcard 31

Q: For which tasks is 2D-aware positional encoding most beneficial?

A: Tasks requiring strong spatial reasoning or resolution transfer, such as detection and segmentation.

Flashcard 32

Q: What efficient idea is used to implement patch embedding conceptually?

A: Use a 2D projection over non-overlapping patches where the projection window and stride both equal the patch size.

Flashcard 33

Q: In patch embedding, what does the embedding dimension D represent?

A: It is the number of output features for each patch token, equivalent to the token embedding dimension.

Flashcard 34

Q: After projecting patches but before reordering into the standard token sequence, what tensor shape is often obtained?

A: [B, D, N], where B is batch size, D is embedding dimension, and N is the number of patches.

Flashcard 35

Q: What is the formula for attention in Multi-Head Self-Attention when d_h is the head dimension?

A: $\text{Attention}(Q, K, V) = \text{softmax}((QK^T)/\sqrt{d_h}) V$.

Flashcard 36

Q: Which activation function is typically used in the MLP block of a Transformer?

A: GELU.

Flashcard 37

Q: What is the sequence length if an image is split into 64 patches and one class token is added?

A: 65 tokens.

Flashcard 38

Q: What is the purpose of weight decay during ViT training?

A: It acts as regularization by penalizing large weight magnitudes.

Flashcard 39

Q: What does FID measure in generative modeling?

A: It measures distributional closeness between generated images and real images in feature space; lower is usually better.

Flashcard 40

Q: How does precision differ from recall in generative-model evaluation?

A: Precision measures realism or fidelity to the real manifold, while recall measures coverage or diversity of the generated distribution.

Flashcard 41

Q: What did comparisons between U-Net and U-ViT indicate using sFID and FID?

A: The ViT-style backbone U-ViT quantitatively outperformed the CNN-style U-Net baseline.

Flashcard 42

Q: In DiT scaling analysis, why can larger models be superior to smaller models with more sampling compute?

A: Smaller models may not close the performance gap even when they use more computation steps at sampling time.

Flashcard 43

Q: Why is truncated-normal initialization often used for class tokens and positional embeddings in ViT?

A: It is a common practice to stabilize training and improve initial convergence.

Flashcard 44

Q: What is the purpose of the scale factor $1/\sqrt{d_h}$ in attention?

A: It prevents dot products from becoming too large, which stabilizes softmax and gradient behavior.

Flashcard 45

Q: In a ViT classifier, which token final hidden state is fed into the linear classification head?

A: The class token, usually index 0 of the output sequence.

Flashcard 46

Q: Concept: patch partitioning.

A: Patch partitioning is the process of dividing an input image into non-overlapping spatial grids that become the fundamental input units for the Transformer.

Flashcard 47

Q: What is the relationship between number of heads and embedding dimension D in multi-head attention?

A: The embedding dimension D must be divisible by the number of heads.

Flashcard 48

Q: Why is bidirectional attention necessary for the class token to function effectively?

A: It allows the class token to attend to all image patches simultaneously and form a global summary.

Flashcard 49

Q: What is Inception Score (IS) used to reward in generative modeling evaluation?

A: Images that are individually recognizable and collectively varied across classes.

Flashcard 50

Q: What is the input-to-output transformation of the MLP in a Transformer block regarding tensor shape?

A: The MLP transforms each token independently and preserves the sequence shape $[B, N+1, D]$.

14. Last-Minute Exam Checklist

- ☐ Can you compute N from H, W, and P?

- ☐ Can you explain why a patch is flattened and projected into D dimensions?
- ☐ Can you explain why positional information is required?
- ☐ Can you explain the class token and why it is optional?
- ☐ Can you write the attention formula and identify Q, K, and V?
- ☐ Can you describe residual connections, LayerNorm, and the MLP/FFN inside a ViT block?
- ☐ Can you compare classification heads and general image embeddings?
- ☐ Can you explain why ViTs connect naturally to CLIP, SigLIP, DiT, U-ViT, and MaskGIT?
- ☐ Can you discuss CNN versus ViT tradeoffs without claiming that one is always better?

End of Chapter 7 updated theory-only study guide.